

Approximative Darstellung reeller Zahlen

Max Recker

max.recker@uni-bielefeld.de

5.5.2026

1 4.1 b-adische Darstellung reeller Zahlen

In diesem Abschnitt geht es um die grundlegende Frage, wie man reelle Zahlen in einem Zahlensystem zur Basis b darstellen kann. Das Dezimalsystem entspricht dabei der Basis $b = 10$, das Binärsystem der Basis $b = 2$.

Die folgende Aussage zeigt, dass jede von Null verschiedene reelle Zahl eine eindeutig bestimmte Darstellung dieser Form besitzt.

Satz 1. *Sei $b \in \mathbb{N}$ mit $b \geq 2$. Sei ferner $x \in \mathbb{R} \setminus \{0\}$. Dann existieren eindeutig bestimmte Zahlen*

$$E \in \mathbb{Z}, \quad \sigma \in \{-1, 1\}, \quad z_i \in \{0, 1, \dots, b-1\}$$

für alle $i \in \mathbb{N} \cup \{0\}$, so dass gilt:

$$\{i \in \mathbb{N} \mid z_i \neq b-1\} \text{ ist unendlich,} \quad z_0 \neq 0,$$

und

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{\infty} z_i b^{-i} \right).$$

Diese Darstellung heißt die (normalisierte) b-adische Darstellung von x .

Bedeutung der einzelnen Bestandteile

Die Formel

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{\infty} z_i b^{-i} \right)$$

setzt sich aus mehreren Teilen zusammen:

- x : die darzustellende reelle Zahl.

- $\sigma \in \{-1, 1\}$: das Vorzeichen der Zahl. Dabei gilt:

$\sigma = 1$ für positive Zahlen, $\sigma = -1$ für negative Zahlen.

- b : die Basis des verwendeten Zahlensystems.
- $E \in \mathbb{Z}$: der Exponent. Er bestimmt die Größenordnung der Zahl.
- z_i : die Ziffern der Darstellung. Jede Ziffer liegt zwischen 0 und $b - 1$.
-

$$\sum_{i=0}^{\infty} z_i b^{-i}$$

ist die eigentliche Zifferndarstellung (Mantisse) der Zahl.

Warum fordert man $z_0 \neq 0$?

Die Bedingung $z_0 \neq 0$ sorgt dafür, dass die Darstellung normalisiert ist. Das bedeutet: Die erste Ziffer darf nicht Null sein. Dadurch wird verhindert, dass dieselbe Zahl mehrere verschiedene Darstellungen besitzt.

Warum muss die Menge $\{i \in \mathbb{N} \mid z_i \neq b - 1\}$ unendlich sein?

Diese Bedingung verhindert Doppeldarstellungen wie im Dezimalsystem:

$$1,0000 \dots = 0,9999 \dots$$

Durch die Forderung, dass nicht ab irgendeinem Punkt nur noch Ziffern $b - 1$ auftreten, wird die Darstellung eindeutig gemacht.

Anschauliche Interpretation

Jede Zahl besteht also aus drei Bausteinen:

$$\text{Zahl} = \text{Vorzeichen} \cdot \text{Größenordnung} \cdot \text{Zifferndarstellung}$$

Das ist genau das Prinzip, das später auch bei Maschinenzahlen und Gleitkommazahlen auf dem Computer verwendet wird.

Wichtige Beispiele für Basen

Die Basis b bestimmt, mit welchen Ziffern eine Zahl dargestellt wird. Es muss stets gelten:

$$b \in \mathbb{N}, \quad b \geq 2.$$

Typische Beispiele sind:

$b = 10$: Dezimalsystem
 $b = 2$: Binärsystem
 $b = 8$: Oktalsystem
 $b = 16$: Hexadezimalsystem

Das im Alltag verwendete Zahlensystem ist das Dezimalsystem. Computer arbeiten intern dagegen hauptsächlich mit dem Binärsystem.

Die Bedeutung der Ziffern z_i

Die Zahl z_i ist die i -te Ziffer der Darstellung von x .
 Beispielsweise bei der Zahl

3,1415

gilt:

$$z_0 = 3, \quad z_1 = 1, \quad z_2 = 4, \quad z_3 = 1, \quad z_4 = 5.$$

Jede Ziffer darf nur aus den erlaubten Symbolen der Basis stammen, also:

$$z_i \in \{0, 1, \dots, b-1\}.$$

Beispiele:

$$\begin{aligned}
 b = 10 &\Rightarrow z_i \in \{0, 1, \dots, 9\}, \\
 b = 2 &\Rightarrow z_i \in \{0, 1\}.
 \end{aligned}$$

Die Bedeutung von b^{-i}

Der Ausdruck b^{-i} gibt den Stellenwert der jeweiligen Ziffer an. Er bestimmt also, wie stark die Ziffer z_i in die Zahl eingeht.

Es gilt:

$$b^{-i} = \frac{1}{b^i}.$$

Im Dezimalsystem ($b = 10$) bedeutet das:

$$\begin{aligned}
 10^0 &= 1 && \text{(Einerstelle)} \\
 10^{-1} &= 0,1 && \text{(Zehntelstelle)} \\
 10^{-2} &= 0,01 && \text{(Hundertstelstelle)} \\
 10^{-3} &= 0,001 && \text{usw.}
 \end{aligned}$$

Zusammenspiel von Ziffer und Stellenwert

Jeder Summand

$$z_i \cdot b^{-i}$$

besteht aus:

Ziffer · Stellenwert.

Er beschreibt also den Beitrag der i -ten Stelle zur gesamten Zahl.

Die komplette Zahl entsteht dann durch das Addieren aller dieser Beiträge:

$$\sum_{i=0}^{\infty} z_i b^{-i}.$$

Bemerkung zur Reihennotation

Wir verwenden im Folgenden die übliche Schreibweise für unendliche Summen (Reihen):

$$\sum_{i=0}^{\infty} a_i := \lim_{n \rightarrow \infty} \sum_{i=0}^n a_i.$$

Das bedeutet:

- Zunächst betrachtet man die endlichen Partialsummen

$$\sum_{i=0}^n a_i.$$

- Anschließend lässt man $n \rightarrow \infty$ gehen.
- Existiert dabei ein Grenzwert, so nennt man diesen Wert die Summe der Reihe.

Im vorliegenden Zusammenhang existiert dieser Grenzwert tatsächlich. Der Grund ist:

- Die Folge der Partialsummen ist monoton wachsend.
- Gleichzeitig ist sie nach oben durch b beschränkt.

Nach dem Monotoniekriterium für Folgen besitzt die Folge daher einen Grenzwert.

Somit ist der Ausdruck

$$\sum_{i=0}^{\infty} z_i b^{-i}$$

wohldefiniert.

Beweis der Aussage

Sei $x \in \mathbb{R} \setminus \{0\}$.

Offensichtlich muss für das Vorzeichen gelten:

$$\sigma = \begin{cases} -1, & \text{falls } x < 0, \\ 1, & \text{falls } x > 0. \end{cases}$$

Für Ziffern

$$z_i \in \{0, 1, \dots, b-1\} \quad (i \in \mathbb{N} \cup \{0\})$$

mit

$$\{i \in \mathbb{N} \mid z_i \neq b-1\} \neq \emptyset \quad \text{und} \quad z_0 \neq 0$$

zeigen wir nun:

$$1 \leq \sum_{i=0}^{\infty} z_i b^{-i} < \sum_{i=0}^{\infty} (b-1) b^{-i} = b.$$

1. Untere Schranke

Da $z_0 \neq 0$ und $z_0 \in \{0, 1, \dots, b-1\}$, folgt:

$$z_0 \geq 1.$$

Somit gilt:

$$\sum_{i=0}^{\infty} z_i b^{-i} = z_0 + \sum_{i=1}^{\infty} z_i b^{-i} \geq z_0 \geq 1.$$

Also:

$$1 \leq \sum_{i=0}^{\infty} z_i b^{-i}.$$

Gleichheit gilt genau dann, wenn alle weiteren Ziffern verschwinden, also

$$z_i = 0 \quad \text{für alle } i \geq 1.$$

2. Obere Schranke

Da jede Ziffer höchstens $b-1$ ist, gilt für alle i :

$$z_i \leq b-1.$$

Daher folgt:

$$\sum_{i=0}^{\infty} z_i b^{-i} \leq \sum_{i=0}^{\infty} (b-1) b^{-i}.$$

Nun berechnen wir die rechte Seite:

$$\sum_{i=0}^{\infty} (b-1) b^{-i} = (b-1) \sum_{i=0}^{\infty} b^{-i}.$$

Dies ist eine geometrische Reihe mit Quotienten $q = \frac{1}{b}$. Also:

$$\sum_{i=0}^{\infty} b^{-i} = \frac{1}{1 - \frac{1}{b}} = \frac{b}{b-1}.$$

Somit:

$$(b-1) \sum_{i=0}^{\infty} b^{-i} = (b-1) \frac{b}{b-1} = b.$$

Also gilt:

$$\sum_{i=0}^{\infty} z_i b^{-i} \leq b.$$

Wegen der Zusatzbedingung

$$\{ i \in \mathbb{N} \mid z_i \neq b-1 \} \neq \emptyset$$

ist jedoch mindestens eine Ziffer echt kleiner als $b-1$. Daher kann die Summe den Wert b nicht erreichen. Also sogar:

$$\sum_{i=0}^{\infty} z_i b^{-i} < b.$$

3. Ergebnis

Insgesamt folgt:

$$1 \leq \sum_{i=0}^{\infty} z_i b^{-i} < b.$$

Damit liegt die Mantisse stets im Intervall $[1, b)$.

Bestimmung des Exponenten und Rekursion für die Ziffern

Aus der Normierungsbedingung

$$1 \leq a_0 < b \quad \text{mit} \quad a_0 = b^{-E}|x|$$

hatten wir bereits gefolgert, dass E so gewählt werden muss, dass

$$b^E \leq |x| < b^{E+1}.$$

Daraus folgt unmittelbar:

$$E = \lfloor \log_b |x| \rfloor.$$

Somit ist der Exponent eindeutig festgelegt.

Nun setzen wir

$$a_0 := b^{-E}|x|$$

und definieren rekursiv für alle $i \in \mathbb{N} \cup \{0\}$:

$$z_i := \lfloor a_i \rfloor \quad \text{und} \quad a_{i+1} := b(a_i - z_i).$$

Bedeutung dieser Definition

Dabei ist

- $z_i = \lfloor a_i \rfloor$ der ganzzahlige Anteil von a_i , also die nächste Ziffer der Darstellung,
- $a_i - z_i$ der verbleibende Nachkommanteil,
- und durch Multiplikation mit b wird dieser Rest so verschoben, dass die nächste Ziffer wieder vorne erscheint.

Gerade deshalb ist die Rekursion so sinnvoll: Mit jedem Schritt „holen“ wir genau eine weitere Ziffer der b -adischen Darstellung nach vorne.

Warum ist z_i tatsächlich eine erlaubte Ziffer?

Da stets

$$1 \leq a_i < b \quad \text{bzw.} \quad 0 \leq a_i < b$$

je nach Rekursionsschritt gilt, liegt der ganzzahlige Anteil automatisch in der Menge

$$z_i \in \{0, 1, \dots, b-1\}.$$

Damit ist sichergestellt, dass jede so gewonnene Ziffer wirklich eine zulässige Ziffer des Zahlensystems zur Basis b ist.

Anschauliches Beispiel im Dezimalsystem

Betrachten wir zur Veranschaulichung die Zahl

$$a_0 = 3,14.$$

Dann erhält man:

$$z_0 = \lfloor 3,14 \rfloor = 3.$$

Der Rest ist:

$$3,14 - 3 = 0,14.$$

Nun wird mit $b = 10$ multipliziert:

$$a_1 = 10 \cdot 0,14 = 1,4.$$

Also:

$$z_1 = \lfloor 1,4 \rfloor = 1.$$

Wieder bleibt ein Rest:

$$1,4 - 1 = 0,4.$$

Dann folgt:

$$a_2 = 10 \cdot 0,4 = 4, \quad z_2 = \lfloor 4 \rfloor = 4.$$

Man erkennt: Die Ziffern 3, 1, 4 werden Schritt für Schritt aus der Zahl herausgelesen.

Genau dieses Verfahren liefert allgemein die Ziffern $z_0, z_1, z_2, \dots$ der b -adischen Darstellung.

Eigenschaften der rekursiv definierten Zahlen a_i und z_i

Man beachte:

$$1 \leq a_0 < b \quad \text{und} \quad 0 \leq a_i < b \quad \text{für alle } i \in \mathbb{N}.$$

Daraus folgt unmittelbar:

$$z_i = \lfloor a_i \rfloor \in \{0, 1, \dots, b-1\} \quad \text{für alle } i,$$

sowie insbesondere

$$z_0 \neq 0.$$

Denn aus $1 \leq a_0 < b$ folgt:

$$1 \leq \lfloor a_0 \rfloor \leq b - 1.$$

Also ist die erste Ziffer stets von Null verschieden.

Warum ist die Normierung wichtig?

Ziel ist es, jede Zahl $x \neq 0$ in der Form

$$x = \sigma \cdot b^E \cdot (\text{Mantisse})$$

darzustellen, wobei die Mantisse die Bedingung

$$1 \leq \text{Mantisse} < b$$

erfüllt.

Diese Bedingung nennt man *Normalisierung*. Sie sorgt dafür, dass die Darstellung eindeutig und systematisch wird.

Beispiel im Dezimalsystem:

$$314 = 3,14 \cdot 10^2.$$

Hier gilt:

$$\sigma = 1, \quad b = 10, \quad E = 2, \quad \text{Mantisse} = 3,14.$$

Die Mantisse liegt tatsächlich im gewünschten Bereich:

$$1 \leq 3,14 < 10.$$

Wie findet man E ?

Man sucht einen Exponenten E , sodass

$$1 \leq \frac{|x|}{b^E} < b.$$

Dann kann man setzen:

$$\text{Mantisse} = \frac{|x|}{b^E}.$$

Somit erhält man:

$$x = \sigma \cdot b^E \cdot \frac{|x|}{b^E} = \sigma \cdot |x| = x.$$

Die Darstellung ist also korrekt.

Warum wird dadurch z_0 automatisch gültig?

Da die Mantisse im Intervall $[1, b)$ liegt, befindet sich ihr ganzzahliger Anteil automatisch in der Menge

$$\{1, 2, \dots, b-1\}.$$

Dieser ganzzahlige Anteil ist genau

$$z_0 = \lfloor a_0 \rfloor.$$

Damit ist die erste Ziffer stets eine erlaubte Ziffer und niemals Null.

Bedeutung von a_0

Man setzt

$$a_0 := b^{-E}|x| = \frac{|x|}{b^E}.$$

Die Zahl a_0 ist also nichts anderes als die normierte Form von x , also die Mantisse, aus der anschließend alle Ziffern $z_0, z_1, z_2, \dots$ gewonnen werden.

Darstellung von a_0 durch Partialsummen und Restglied

Wir behaupten:

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1} \quad \text{für alle } n \in \mathbb{N} \cup \{0\}.$$

Bedeutung der Formel

Diese Gleichung zerlegt a_0 in zwei Teile:

$$a_0 = \underbrace{\sum_{i=0}^n z_i b^{-i}}_{\text{bereits bestimmte Ziffern}} + \underbrace{a_{n+1} b^{-n-1}}_{\text{Restglied}}.$$

- Die Summe enthält alle bisher berechneten Ziffern $z_0, \dots, z_n$.
- Das Restglied enthält den noch nicht ausgeschriebenen Teil der Zahl.
- Mit wachsendem n wird dieser Rest immer kleiner.

Am Ende wollen wir daraus erhalten:

$$a_0 = \sum_{i=0}^{\infty} z_i b^{-i}.$$

Die unendliche Reihe ist also die exakte Darstellung von a_0 .

Hinweis: Falls bereits

$$1 \leq x < b$$

gilt, dann ist keine zusätzliche Normierung nötig und es gilt direkt:

$$a_0 = x.$$

Beweis durch vollständige Induktion über n

Induktionsanfang: $n = 0$ Zu zeigen ist:

$$a_0 = z_0 b^0 + a_1 b^{-1}.$$

Da $b^0 = 1$, ist dies gleichbedeutend mit:

$$a_0 = z_0 + \frac{a_1}{b}.$$

Aus der Rekursionsvorschrift

$$a_1 = b(a_0 - z_0)$$

folgt:

$$\frac{a_1}{b} = a_0 - z_0.$$

Einsetzen ergibt:

$$a_0 = z_0 + (a_0 - z_0) = a_0.$$

Damit stimmt die Behauptung für $n = 0$.

✓

Induktionsschritt Sei nun $n \in \mathbb{N}$.

Wir nehmen an, die Behauptung gelte bereits für $n - 1$, also:

$$a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + a_n b^{-n}.$$

Zu zeigen ist:

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}.$$

Aus der Rekursionsvorschrift

$$a_{n+1} = b(a_n - z_n)$$

folgt durch Umformen:

$$\frac{a_{n+1}}{b} = a_n - z_n$$

und damit

$$a_n = z_n + \frac{a_{n+1}}{b}.$$

Nun setzen wir dies in die Induktionsannahme ein:

$$a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + \left(z_n + \frac{a_{n+1}}{b} \right) b^{-n}.$$

Ausmultiplizieren liefert:

$$a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + z_n b^{-n} + \frac{a_{n+1}}{b} b^{-n}.$$

Wegen

$$\frac{1}{b} b^{-n} = b^{-n-1}$$

erhält man:

$$a_0 = \sum_{i=0}^{n-1} z_i b^{-i} + z_n b^{-n} + a_{n+1} b^{-n-1}.$$

Die ersten beiden Summanden lassen sich zusammenfassen zu:

$$\sum_{i=0}^n z_i b^{-i}.$$

Somit folgt:

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}.$$

Damit ist der Induktionsschritt gezeigt.

✓

Schlussfolgerung

Nach dem Prinzip der vollständigen Induktion gilt somit für alle $n \in \mathbb{N} \cup \{0\}$:

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1}.$$

Übergang zur unendlichen Darstellung

Aus der bereits bewiesenen Gleichung

$$a_0 = \sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1} \quad \text{für alle } n \in \mathbb{N} \cup \{0\}$$

folgt nun durch Grenzübergang $n \rightarrow \infty$:

$$a_0 = \lim_{n \rightarrow \infty} \left(\sum_{i=0}^n z_i b^{-i} + a_{n+1} b^{-n-1} \right).$$

Um daraus die gewünschte unendliche Darstellung zu erhalten, müssen wir zeigen, dass das Restglied verschwindet, also:

$$a_{n+1} b^{-n-1} \rightarrow 0 \quad (n \rightarrow \infty).$$

Dann bleibt im Grenzwert nur noch:

$$a_0 = \lim_{n \rightarrow \infty} \sum_{i=0}^n z_i b^{-i} = \sum_{i=0}^{\infty} z_i b^{-i}.$$

Damit ergibt sich schließlich:

$$x = \sigma \cdot b^E \cdot a_0 = \sigma \cdot b^E \cdot \sum_{i=0}^{\infty} z_i b^{-i}.$$

Warum verschwindet das Restglied?

Es wurde bereits gezeigt, dass für alle Indizes gilt:

$$1 \leq a_{n+1} < b.$$

Daraus folgt:

$$0 \leq a_{n+1} b^{-n-1} < b \cdot b^{-n-1} = b^{-n}.$$

Da $b \geq 2$ ist, gilt:

$$b^{-n} \rightarrow 0 \quad (n \rightarrow \infty).$$

Somit liegt die Folge $(a_{n+1} b^{-n-1})$ zwischen 0 und einer Folge, die gegen 0 konvergiert. Nach dem Sandwichprinzip folgt daher:

$$a_{n+1} b^{-n-1} \rightarrow 0.$$

Schluss

Damit ist gezeigt:

$$a_0 = \sum_{i=0}^{\infty} z_i b^{-i}$$

und somit insgesamt

$$x = \sigma \cdot b^E \cdot \left(\sum_{i=0}^{\infty} z_i b^{-i} \right).$$

Das ist genau die behauptete normalisierte b -adische Darstellung von x .

Eindeutigkeit der Darstellung

Nun bleibt noch zu zeigen, dass die Darstellung eindeutig ist.

Die Zusatzbedingung

$$\{ i \in \mathbb{N} \mid z_i \neq b-1 \} \text{ ist unendlich}$$

verhindert Mehrfachdarstellungen wie im Dezimalsystem

$$1,0000 \dots = 0,9999 \dots$$

oder allgemein das Auftreten eines unendlichen $(b-1)$ -Schwanzes.

Warum ist diese Bedingung nötig?

Angenommen,

$$\{ i \in \mathbb{N} \mid z_i \neq b-1 \}$$

wäre endlich. Dann gäbe es also ein $n_0 \in \mathbb{N}$, sodass

$$z_i = b-1 \quad \text{für alle } i > n_0.$$

Dann hätte die Mantisse die Form

$$a_0 = \sum_{i=0}^{n_0} z_i b^{-i} + \sum_{i=n_0+1}^{\infty} (b-1) b^{-i}.$$

Wir berechnen den zweiten Summanden:

$$\sum_{i=n_0+1}^{\infty} (b-1) b^{-i} = (b-1) \sum_{i=n_0+1}^{\infty} b^{-i}.$$

Dies ist wieder eine geometrische Reihe. Ausklammern von $b^{-(n_0+1)}$ ergibt:

$$(b-1)b^{-(n_0+1)} \sum_{k=0}^{\infty} b^{-k}.$$

Mit

$$\sum_{k=0}^{\infty} b^{-k} = \frac{1}{1 - \frac{1}{b}} = \frac{b}{b-1}$$

folgt:

$$(b-1)b^{-(n_0+1)} \frac{b}{b-1} = b^{-n_0}.$$

Also gilt:

$$\sum_{i=n_0+1}^{\infty} (b-1)b^{-i} = b^{-n_0}.$$

Damit wäre

$$a_0 = \sum_{i=0}^{n_0} z_i b^{-i} + b^{-n_0}.$$

Das bedeutet: Man könnte die Ziffer an Stelle n_0 um 1 erhöhen und danach nur noch Nullen schreiben. Es gäbe also eine zweite Darstellung derselben Zahl.

Genau deshalb wird der Fall eines endlosen $(b-1)$ -Schwanzes ausgeschlossen.

Der eigentliche Widerspruch

Aus der Annahme, dass ab einem gewissen Index nur noch Ziffern $b-1$ auftreten, folgt nach der obigen Rechnung:

$$a_{n_0+1} = b.$$

Dies ist jedoch unmöglich.

Denn aus der Rekursion wurde bereits gezeigt, dass für alle Restzahlen gilt:

$$1 \leq a_n < b \quad \text{bzw.} \quad 0 \leq a_n < b.$$

Insbesondere darf also niemals

$$a_{n_0+1} = b$$

gelten.

Damit entsteht ein Widerspruch zur Annahme, dass ab irgendeinem Punkt nur noch die Ziffer $b-1$ vorkommt.

Also muss die Annahme falsch sein. Folglich ist die Menge

$$\{i \in \mathbb{N} \mid z_i \neq b-1\}$$

nicht endlich, sondern unendlich.

Bedeutung

Es gibt also unendlich viele Stellen, an denen die Ziffer nicht gleich $b-1$ ist. Genau diese Bedingung verhindert doppelte Darstellungen und macht die normalisierte b -adische Darstellung eindeutig.

2. Eindeutigkeit der Darstellung

Es bleibt die Eindeutigkeit der Ziffernfolge zu zeigen. Die Eindeutigkeit von Vorzeichen σ und Exponent E wurde bereits gezeigt.

Angenommen, eine Zahl $x \neq 0$ besitze zwei verschiedene normalisierte b -adische Darstellungen:

$$x = \sigma \cdot b^E \sum_{i=0}^{\infty} y_i b^{-i} = \sigma \cdot b^E \sum_{i=0}^{\infty} z_i b^{-i},$$

wobei beide Ziffernfolgen dieselben Bedingungen erfüllen.

Das bedeutet: Dieselbe Zahl hätte zwei verschiedene Folgen von Ziffern.

Da die Folgen verschieden sind, gibt es einen ersten Index, an dem sie sich unterscheiden. Sei also

$$n := \min\{i \in \mathbb{N}_0 \mid y_i \neq z_i\}.$$

Dann gilt für alle $i < n$:

$$y_i = z_i.$$

Ohne Beschränkung der Allgemeinheit dürfen wir annehmen:

$$y_n < z_n.$$

Da y_n und z_n ganze Ziffern sind, folgt sogar:

$$y_n + 1 \leq z_n.$$

Damit beginnt an der Stelle n der eigentliche Unterschied der beiden Darstellungen.

Da beide Darstellungen gleich sind, können wir durch den gemeinsamen Faktor $\sigma \cdot b^E \neq 0$ teilen. Somit folgt:

$$\sum_{i=0}^{\infty} y_i b^{-i} = \sum_{i=0}^{\infty} z_i b^{-i}.$$

Nun zerlegen wir beide Reihen an der ersten unterschiedlichen Stelle n .

Für die linke Seite gilt:

$$\sum_{i=0}^{\infty} y_i b^{-i} = \sum_{i=0}^{n-1} y_i b^{-i} + y_n b^{-n} + \sum_{i=n+1}^{\infty} y_i b^{-i}.$$

Für die rechte Seite entsprechend:

$$\sum_{i=0}^{\infty} z_i b^{-i} = \sum_{i=0}^{n-1} z_i b^{-i} + z_n b^{-n} + \sum_{i=n+1}^{\infty} z_i b^{-i}.$$

Da für alle $i < n$ bereits $y_i = z_i$ gilt, sind die ersten Summen identisch und heben sich weg.

Es bleibt also:

$$y_n b^{-n} + \sum_{i=n+1}^{\infty} y_i b^{-i} = z_n b^{-n} + \sum_{i=n+1}^{\infty} z_i b^{-i}.$$

Wegen $y_n + 1 \leq z_n$ gilt insbesondere

$$y_n \leq z_n - 1.$$

Damit ist die linke Seite an der Stelle n mindestens um eine Einheit kleiner als die rechte Seite. Im nächsten Schritt zeigen wir, dass die restlichen Nachkommastellen diesen Unterschied nicht mehr ausgleichen können.

Da jede Ziffer höchstens $b - 1$ ist, gilt für den Rest der linken Darstellung:

$$\sum_{i=n+1}^{\infty} y_i b^{-i} \leq \sum_{i=n+1}^{\infty} (b - 1) b^{-i}.$$

Außerdem hatten wir bereits

$$y_n \leq z_n - 1$$

gezeigt, also

$$y_n b^{-n} \leq (z_n - 1) b^{-n}.$$

Daher ist die gesamte linke Seite höchstens

$$\sum_{i=0}^{n-1} y_i b^{-i} + (z_n - 1) b^{-n} + \sum_{i=n+1}^{\infty} (b - 1) b^{-i}.$$

Nun berechnen wir wieder die geometrische Reihe:

$$\sum_{i=n+1}^{\infty} (b - 1) b^{-i} = b^{-n}.$$

Somit wird der obige Ausdruck zu

$$\sum_{i=0}^{n-1} y_i b^{-i} + (z_n - 1)b^{-n} + b^{-n}.$$

Die letzten beiden Summanden vereinfachen sich zu

$$(z_n - 1)b^{-n} + b^{-n} = z_n b^{-n}.$$

Also folgt insgesamt:

$$\sum_{i=0}^{\infty} y_i b^{-i} \leq \sum_{i=0}^{n-1} y_i b^{-i} + z_n b^{-n}.$$

Da für $i < n$ gilt $y_i = z_i$, erhalten wir:

$$\sum_{i=0}^{\infty} y_i b^{-i} \leq \sum_{i=0}^{n-1} z_i b^{-i} + z_n b^{-n}.$$

Andererseits ist wegen $z_i \geq 0$ für alle $i > n$:

$$\sum_{i=0}^{n-1} z_i b^{-i} + z_n b^{-n} \leq \sum_{i=0}^{\infty} z_i b^{-i}.$$

Zusammen also:

$$\sum_{i=0}^{\infty} y_i b^{-i} \leq \sum_{i=0}^{\infty} z_i b^{-i}.$$

Tatsächlich gilt sogar strikte Ungleichung. Denn Gleichheit könnte nur dann eintreten, wenn in der linken Darstellung ab der Stelle $n + 1$ nur noch Ziffern $b - 1$ auftreten würden. Genau dieser Fall ist jedoch durch die Bedingung ausgeschlossen, dass es unendlich viele Indizes mit Ziffern ungleich $b - 1$ gibt.

Also folgt:

$$\sum_{i=0}^{\infty} y_i b^{-i} < \sum_{i=0}^{\infty} z_i b^{-i}.$$

Das ist ein Widerspruch zur Annahme, dass beide Reihen dieselbe Zahl darstellen.

Folglich können zwei verschiedene Ziffernfolgen nicht dieselbe normalisierte b -adische Darstellung erzeugen. Die Ziffernfolge ist also eindeutig bestimmt.

Damit ist insgesamt gezeigt:

- Die normalisierte b -adische Darstellung existiert.
- Sie ist eindeutig.

Damit ist Satz 4.1 vollständig bewiesen.

Merksatz und Beispiele

Merksatz:

Wenn an der ersten unterschiedlichen Stelle eine Ziffer kleiner ist, dann kann der restliche unendliche Teil diesen Unterschied nie mehr vollständig ausgleichen.

Beispiele für $b = 2$:

1. Darstellung von $\frac{1}{3}$

$$\frac{1}{3} = 2^{-2} (2^0 + 2^{-2} + 2^{-4} + 2^{-6} + \dots) = (0, \overline{01})_2$$

Das bedeutet: In der Binärdarstellung wiederholt sich das Muster 01 unendlich oft.

2. Darstellung von $8,1$

$$8,1 = 2^3 (2^0 + 2^{-7} + 2^{-8} + 2^{-11} + 2^{-12} + \dots) = (1000, \overline{00011})_2$$

Hier wiederholt sich nach dem Komma periodisch das Muster 0011.

Hinweis:

Solche periodischen Darstellungen kennt man auch aus dem Dezimalsystem, z. B.

$$\frac{1}{3} = 0, \overline{3}.$$

Je nach Basis entstehen also endliche oder unendliche periodische Darstellungen.

2 4.2 Maschinenzahlen

Da in einem Rechner nur endlich viele Stellen einer reellen Zahl dargestellt werden können, verwendet man in Anlehnung an Satz 4.1 (normalisierte b -adische Darstellung) eine endliche, auf Maschinen darstellbare Form.

Während bei reellen Zahlen im Allgemeinen unendlich viele Ziffern auftreten können, muss ein Computer diese Darstellung auf endlich viele Stellen begrenzen. Genau daraus entsteht der Begriff der *Maschinenzahl*.

Man benutzt also dieselbe Grundidee wie bei der b -adischen Darstellung, speichert jedoch nur endlich viele Ziffern der Mantisse.

Die folgende Definition formalisiert diesen Gedanken.

Definition 2. Seien $b, m \in \mathbb{N}$ mit $b \geq 2$.

Eine b -adische, m -stellige, normalisierte Gleitkommazahl hat die Form

$$x = \sigma \cdot b^E \left(\sum_{i=0}^{m-1} z_i b^{-i} \right),$$

wobei gilt:

- $\sigma \in \{-1, 1\}$ ist das Vorzeichen,
- $E \in \mathbb{Z}$ ist der Exponent,
- $z_i \in \{0, 1, \dots, b-1\}$ für alle $i \in \{0, 1, \dots, m-1\}$,
- $z_0 \neq 0$.

Die Summe

$$\sum_{i=0}^{m-1} z_i b^{-i}$$

heißt die Mantisse der Zahl.

Erläuterung

Im Vergleich zur Darstellung reeller Zahlen aus Satz 4.1 besteht der einzige wesentliche Unterschied darin, dass die Zifferndarstellung nicht mehr unendlich viele Stellen besitzt, sondern nach genau m Stellen endet.

Es werden also nur die Ziffern

$$z_0, z_1, \dots, z_{m-1}$$

gespeichert.

Warum endet die Summe bei $m-1$?

Da bei $i=0$ bereits die erste Stelle beginnt, ist:

$$i=0 \text{ erste Stelle, } i=1 \text{ zweite Stelle, } \dots, i=m-1 \text{ m-te Stelle.}$$

Genau deshalb läuft der Index bis $m-1$ und nicht bis m .

Der Grundgedanke lautet: Ein Rechner kann nur endlich viele Stellen speichern. Deshalb arbeitet man mit endlich vielen Mantissenziffern.

Beispiele

1. Beispiel: Dezimalsystem mit $b = 10$

Für $b = 10$ besitzt die Zahl

$$136,5$$

die normalisierte 5-stellige Darstellung

$$136,5 = 1,3650 \cdot 10^2.$$

Die Mantisse hat hier genau fünf Stellen:

$$1, 3, 6, 5, 0.$$

In der wissenschaftlichen Schreibweise bzw. in C++-Notation schreibt man dies oft als:

$$1.3650\text{e}+2$$

Dabei bedeutet **e+2** genau den Faktor 10^2 .

2. Beispiel: Die Zahl $\frac{1}{3}$

Die Zahl

$$\frac{1}{3} = 0,3333 \dots$$

besitzt für $b = 10$ und beliebiges $m \in \mathbb{N}$ keine exakte Darstellung als m -stellige normalisierte Gleitkommazahl.

Der Grund ist:

- Die Dezimaldarstellung ist unendlich periodisch.
- Eine Maschinenzahl besitzt aber nur endlich viele Mantissenziffern.
- Daher kann $\frac{1}{3}$ nicht exakt gespeichert, sondern nur angenähert werden.

Zum Beispiel:

$$\frac{1}{3} \approx 0,33333$$

bei einer 5-stelligen Mantisse.

3. Beispiel: Die Zahl 0

Die Zahl

$$0$$

besitzt keine Darstellung als normalisierte Gleitkommazahl in obiger Form. Der Grund ist die Bedingung

$$z_0 \neq 0.$$

Bei $x = 0$ wäre jedoch jede Mantisse gleich 0, also wäre insbesondere die erste Ziffer ebenfalls 0. Das widerspricht der Normalisierung.

Deshalb wird die Zahl 0 in realen Computersystemen separat behandelt.

Maschinenzahlbereich

Für die Darstellung von Zahlen im Rechner müssen geeignete Werte für

$$b, \quad m, \quad E_{\min}, \quad E_{\max}$$

gewählt werden.

Dabei bedeuten:

- b : Basis des Zahlensystems,
- m : Anzahl der Mantissenstellen,
- $E_{\min}$: kleinster erlaubter Exponent,
- $E_{\max}$: größter erlaubter Exponent.

Der zulässige Exponentenbereich ist also:

$$E \in \{E_{\min}, E_{\min} + 1, \dots, E_{\max}\}.$$

Diese Wahl bestimmt den sogenannten Maschinenzahlbereich

$$F(b, m, E_{\min}, E_{\max}).$$

Definition:

Der Maschinenzahlbereich ist die Menge aller b -adischen, m -stelligen, normalisierten Gleitkommazahlen mit Exponenten

$$E \in \{E_{\min}, \dots, E_{\max}\},$$

zusätzlich der Zahl 0.

Formal also:

$$F(b, m, E_{\min}, E_{\max}) = \{0\} \cup \left\{ \sigma \cdot b^E \left(\sum_{i=0}^{m-1} z_i b^{-i} \right) \left| \begin{array}{l} \sigma \in \{-1, 1\}, \\ E_{\min} \leq E \leq E_{\max}, \\ z_0 \neq 0, \\ z_i \in \{0, \dots, b-1\} \end{array} \right. \right\}.$$

Die Elemente eines solchen (endlichen) Maschinenzahlbereichs heißen *Maschinenzahlen*.

Beispiel

Wählt man etwa

$$b = 10, \quad m = 3, \quad E \in \{-1, 0, 1\},$$

so enthält der Maschinenzahlbereich genau alle normalisierten Dezimalzahlen mit drei Mantissenstellen und diesen erlaubten Exponenten, sowie zusätzlich die Zahl 0.

Weitere Beispiele für Maschinenzahlen

Für das Beispiel

$$b = 10, \quad m = 3, \quad E \in \{-1, 0, 1\}$$

sind unter anderem folgende Zahlen im Maschinenzahlbereich enthalten:

$$1,23 \cdot 10^1 = 12,3,$$

$$9,99 \cdot 10^{-1} = 0,999,$$

$$-4,56 \cdot 10^0 = -4,56,$$

sowie natürlich auch

$$0.$$

Der IEEE-754-Standard

Im Jahr 1985 wurde der IEEE-754-Standard veröffentlicht. Er ist bis heute der wichtigste Standard zur Darstellung von Gleitkommazahlen auf Computern.

IEEE steht für:

Institute of Electrical and Electronics Engineers

Darin wird beispielsweise der Datentyp `double` beschrieben. Für diesen Typ gilt:

$$b = 2, \quad m = 53, \quad E \in \{-1022, \dots, 1023\}.$$

Man schreibt dafür kurz:

$$F_{\text{double}} := F(2, 53, -1022, 1023).$$

Bedeutung dieser Werte

- $b = 2$: Der Rechner arbeitet binär, also mit den Ziffern 0 und 1.
- $m = 53$: Es stehen 53 signifikante Binärstellen für die Genauigkeit zur Verfügung.
- Großer Exponentenbereich: Dadurch können sowohl sehr kleine als auch sehr große Zahlen dargestellt werden.

Speicherbedarf von double

Zur Darstellung einer `double`-Zahl genügen 64 Bit:

- 1 Bit für das Vorzeichen,
- 52 gespeicherte Bits für die Mantisse,
- 11 Bits für den Exponenten.

(Intern kommt durch eine spezielle Normalisierung effektiv eine Präzision von 53 Binärstellen zustande.)

Das sogenannte Hidden Bit

Bei normalisierten Binärzahlen gilt wegen der Normierung stets:

$$z_0 = 1.$$

Das erste Bit der Mantisse ist also immer gleich 1 und muss daher nicht extra gespeichert werden.

Man lässt dieses führende Bit weg. Deshalb nennt man es das *hidden bit* (verstecktes Bit).

Dadurch erklärt sich auch, warum beim Datentyp `double`

$$m = 53$$

gilt, aber nur 52 Bits für die Mantisse gespeichert werden:

- 1 führendes Bit ist immer bekannt ($= 1$),
- nur die restlichen 52 Bits müssen tatsächlich gespeichert werden.

Warum ist das wichtig?

Die endliche Speicherung von Zahlen erklärt viele typische Phänomene der Computerarithmetik:

- Rundungsfehler,
- manche Zahlen (z. B. 0,1) sind nicht exakt darstellbar,
- sehr große Zahlen können zu Overflow führen,
- sehr kleine Zahlen können zu Underflow führen.

Wenn eine Zahl nicht exakt dargestellt werden kann, wird sie durch eine nahegelegene Maschinenzahl ersetzt.

Diese Themen werden in der numerischen Mathematik ausführlich untersucht.

Bias-Darstellung des Exponenten

Für den Exponenten wird in vielen Computersystemen nicht direkt der mathematische Wert E gespeichert, sondern die sogenannte *Bias-Darstellung* verwendet.

Dabei wird eine feste Konstante (der Bias) zum Exponenten addiert.

$$\text{gespeicherter Wert} = E + \text{Bias}.$$

Der Vorteil ist, dass dadurch nur nichtnegative Bitmuster gespeichert werden müssen.

Warum macht man das?

Der mathematische Exponent kann sowohl negativ als auch positiv sein:

$$E \in \{-1022, \dots, 1023\}.$$

Beispiele:

$$2^{-3}, \quad 2^5.$$

Bits bestehen jedoch nur aus 0 und 1. Mit der Bias-Darstellung lässt sich der gesamte Exponentenbereich bequem als nichtnegative Zahl codieren.

Beispiel beim Typ `double`

Beim IEEE-754-`double` stehen 11 Bits für den Exponenten zur Verfügung.

Damit lassen sich die Werte

$$0, \dots, 2047$$

speichern.

Der verwendete Bias ist:

$$1023.$$

Das liegt genau in der Mitte des Bereichs und erlaubt die Codierung negativer und positiver Exponenten.

Gespeichert wird also nicht E , sondern:

$$E + 1023.$$

Beispiele:

$$E = 0 \quad \longrightarrow \quad 1023$$

$$E = 5 \quad \longrightarrow \quad 1028$$

$$E = -3 \quad \longrightarrow \quad 1020$$

Zusammenfassung

Die Bias-Darstellung ist ein technischer Trick, um Exponenten mit normalen Bitmustern speichern zu können, ohne separate Vorzeichenbits für den Exponenten zu benötigen.

Hinweis zu C++ und `double`

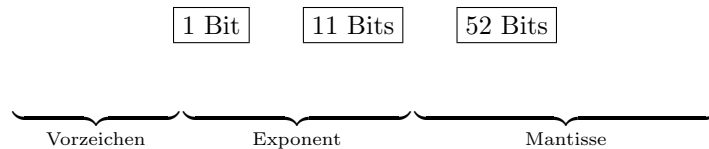
Für den C++-Datentyp `double` schreibt der C++-Standard nicht exakt vor, wie die interne Speicherung technisch umgesetzt werden muss.

In der Praxis wird `double` jedoch auf den meisten Systemen gemäß dem IEEE-754-Standard implementiert.

Davon gehen wir im Folgenden aus.

Bitaufbau einer normalisierten double-Zahl

Die Speicherung in Bits sieht dann wie folgt aus:



Inhalt der drei Bereiche:

$$\text{Vorzeichenbit: } \frac{1 - \sigma}{2}$$

$$\text{Exponentbits: } E + 1023$$

$$\text{Mantissenbits: } z_1, z_2, \dots, z_{52}$$

Bedeutung des Vorzeichenbits

Da

$$\sigma \in \{-1, 1\}$$

gilt:

$$\sigma = 1 \quad \Rightarrow \quad \frac{1 - \sigma}{2} = 0$$

$$\sigma = -1 \quad \Rightarrow \quad \frac{1 - \sigma}{2} = 1$$

Also bedeutet:

- Bit 0: positive Zahl
- Bit 1: negative Zahl

Zusammenfassung

Eine normalisierte `double`-Zahl wird also gespeichert als:

Vorzeichen | Exponent mit Bias | Mantissenbits.

Das ergibt insgesamt:

$$1 + 11 + 52 = 64 \text{ Bit.}$$

Spezielle Exponentenwerte bei `double`

Die beiden freien Exponentenwerte

$$-1023 \quad \text{und} \quad 1024$$

(entsprechend den gespeicherten Bitmustern 0 und 2047 in Bias-Darstellung)
werden nicht für normale Zahlen verwendet.
Stattdessen dienen sie zur Darstellung spezieller Werte.

1. Unendlichkeit $\pm\infty$

Wenn das Ergebnis einer Rechnung größer ist als die größte darstellbare Zahl, entsteht:

$$+\infty \quad \text{oder} \quad -\infty$$

je nach Vorzeichen.
Beispiel:

$$10^{5000}$$

ist für den Typ `double` viel zu groß.
Das Ergebnis wird daher als Unendlichkeit gespeichert.
Dies nennt man *Overflow*.

2. Sehr kleine Zahlen

Sehr kleine Zahlen, deren Betrag kleiner ist als die kleinste normalisierte Zahl, werden zunächst als sogenannte *subnormale Zahlen* dargestellt.
Sind Zahlen noch kleiner, entsteht schließlich:

$$0.$$

Dieser Effekt heißt *Underflow*.

3. NaN (Not a Number)

Manche Rechenoperationen liefern kein sinnvolles reelles Ergebnis.
Beispiel:

$$\log(-1)$$

(im Bereich der reellen Zahlen).
Dann verwendet das System den speziellen Wert:

NaN

Das steht für *Not a Number*.

Der Vorteil: Programme können weiterlaufen, ohne sofort abzustürzen.

Zusammenfassung

Die reservierten Exponentenwerte werden also verwendet für:

- $+\infty$ und $-\infty$,
- subnormale Zahlen,
- Underflow,
- NaN.

Größte darstellbare Zahl vom Typ double

Die größte endlich darstellbare Zahl im Bereich F_{double} ist

$$\max F_{\text{double}} = \text{std::numeric_limits<double>::max()}.$$

Sie entsteht durch:

- größtmöglichen Exponenten $E = 1023$,
- größtmögliche Mantisse.

Die größte Mantisse ist:

$$\left(1, \underbrace{11 \dots 1}_{52 \text{ Einsen}}\right)_2 = 2 - 2^{-52}.$$

Daher gilt:

$$\max F_{\text{double}} = (2 - 2^{-52}) \cdot 2^{1023}.$$

Ausmultipliziert:

$$(2 - 2^{-52})2^{1023} = 2^{1024} - 2^{971}.$$

Numerisch ergibt sich:

$$\max F_{\text{double}} \approx 1,797693 \cdot 10^{308}.$$

Kleinste positive normalisierte Zahl

Die kleinste positive normalisierte Maschinenzahl in F_{double} ist

$$\min\{f \in F_{\text{double}} \mid f > 0\} = \text{std::numeric_limits}<\text{double}>::\text{min}().$$

Sie entsteht durch:

- kleinsten normalen Exponenten $E = -1022$,
- kleinste normierte Mantisse $1,0_2$.

Also:

$$\min = 1,0 \cdot 2^{-1022}.$$

Numerisch:

$$\min \approx 2,225074 \cdot 10^{-308}.$$

Wichtig: Dies ist die kleinste *normalisierte* positive Zahl. Durch subnormale Zahlen können noch kleinere positive Werte dargestellt werden.

Der Darstellungsbereich $\text{range}(F)$

Wir bezeichnen mit

$$\text{range}(F) := [\min\{f \in F \mid f > 0\}, \max F]$$

das kleinste Intervall, in dem alle darstellbaren positiven Zahlen liegen.

Es beschreibt also den Größenbereich der positiven Maschinenzahlen eines Zahlensystems.

Vergleich wichtiger C++-Datentypen

1. float

Der C++-Datentyp `float` (von *floating point*) verwendet meist nur 32 Bit.

Dadurch besitzt er:

- einen kleineren Darstellungsbereich,
- eine geringere Genauigkeit,
- dafür oft weniger Speicherverbrauch.

Er ist deshalb für numerische Berechnungen häufig weniger geeignet als `double`.

2. `double`

Der Datentyp `double` ist in C++ der Standardtyp für die approximative Darstellung reeller Zahlen.

Er bietet einen guten Kompromiss aus:

- Genauigkeit,
- großem Wertebereich,
- Rechengeschwindigkeit.

Darum wird er in wissenschaftlichen Programmen sehr häufig verwendet.

3. `long double`

Der Datentyp `long double` besitzt auf vielen Systemen eine höhere Genauigkeit als `double`.

Häufig werden intern 80 Bit verwendet, zum Beispiel:

- 64 Bit für die Mantisse,
- 16 Bit für den Exponenten.

(Die genaue Implementierung hängt jedoch vom System und Compiler ab.)

Rolle der FPU

Viele Prozessoren besitzen eine FPU (*Floating Point Unit*).

Das ist eine spezielle Recheneinheit für Gleitkommaoperationen.

Sie kann elementare Operationen wie

+, −, ·, /

auf Gleitkommazahlen sehr schnell ausführen.

Merksatz

Für die meisten numerischen Anwendungen ist `double` die beste Standardwahl.

3 4.3 Rundungen

Da es nur endlich viele Maschinenzahlen gibt, können reelle Zahlen im Allgemeinen nicht immer exakt dargestellt werden.

Man muss sich daher mit gerundeten Darstellungen zufriedengeben.

Die folgende Definition beschreibt mathematisch, was unter einer Rundung auf einen Maschinenzahlbereich verstanden wird.

Definition 3. *Sei*

$$F = F(b, m, E_{\min}, E_{\max})$$

ein Maschinenzahlbereich.

Eine Abbildung

$$\text{rd} : \mathbb{R} \rightarrow F$$

heißt Rundung (zu F), wenn für alle $x \in \mathbb{R}$ gilt:

$$|x - \text{rd}(x)| = \min_{a \in F} |x - a|.$$

Bedeutung der Definition

Die Rundungsfunktion rd ordnet jeder reellen Zahl x eine Maschinenzahl aus F zu.

Dabei wird genau diejenige Maschinenzahl gewählt, die den kleinsten Abstand zu x besitzt.

Mit anderen Worten:

$$\text{rd}(x)$$

ist die darstellbare Zahl, die am nächsten bei x liegt.

Interpretation der Formel

Die Gleichung

$$|x - \text{rd}(x)| = \min_{a \in F} |x - a|$$

bedeutet:

- Links steht der Abstand zwischen der echten Zahl x und ihrer gerundeten Darstellung.
- Rechts steht der kleinste mögliche Abstand zu irgendeiner Maschinenzahl $a \in F$.
- Die Rundung liefert also stets die bestmögliche Approximation innerhalb von F .

Warum ist das wichtig?

Diese Definition ist besonders wichtig, wenn eine Zahl nicht selbst im Maschinenzahlbereich liegt.

Dann muss sie durch eine nahegelegene Maschinenzahl ersetzt werden.

Genau dieser Vorgang passiert bei numerischen Berechnungen auf dem Computer ständig.

Mehrdeutige Rundungen

Man beachte, dass es mehrere mögliche Rundungsvorschriften geben kann.

Es kann nämlich vorkommen, dass eine Zahl genau in der Mitte zwischen zwei Maschinenzahlen liegt.

Dann besitzen beide Zahlen denselben Abstand zu x .

Beispiel

Im Maschinenzahlbereich

$$F(10, 2, 0, 2)$$

liegt die Zahl

$$12,5$$

genau zwischen

$$12 \quad \text{und} \quad 13.$$

Daher könnte man sowohl auf

$$12$$

als auch auf

$$13$$

runden.

Kaufmännisches Runden

Beim üblichen kaufmännischen Runden wird im Zweifelsfall auf die betragsmäßig größere Zahl gerundet.

Also zum Beispiel:

$$12,5 \mapsto 13.$$

IEEE-754-Regel: Runden zur geraden Zahl

Der IEEE-754-Standard verwendet jedoch eine andere Regel:

Im Zweifelsfall wird so gerundet, dass die letzte gespeicherte Stelle gerade ist.

Man nennt dies auch:

Round to nearest, ties to even

Beispiele:

$$\text{rd}(12,5) = 12, \quad \text{rd}(13,5) = 14.$$

Ebenso:

$$2,25 \mapsto 2,2, \quad 2,35 \mapsto 2,4.$$

Denn die Endziffern 2 und 4 sind gerade.

Warum macht man das?

Diese Regel wurde gewählt, weil sie Rundungsfehler langfristig besser ausgleicht.

Bei vielen aufeinanderfolgenden Rechnungen entsteht dadurch im Mittel ein kleinerer systematischer Fehler als beim ständigen Aufrunden.

Das ist besonders wichtig in numerischen Verfahren und bei sehr vielen Berechnungen.

Absolute und relative Fehler

Bei Näherungen möchte man messen, wie stark ein approximierter Wert vom exakten Wert abweicht.

Dazu verwendet man den absoluten und den relativen Fehler.

Definition 4. *Es seien*

$$x, \tilde{x} \in \mathbb{R},$$

wobei $\tilde{x}$ eine Näherung von x sei.

Dann heißt

$$|x - \tilde{x}|$$

der absolute Fehler.

Falls $x \neq 0$ heißt

$$\left| \frac{x - \tilde{x}}{x} \right|$$

der relative Fehler.

Bedeutung

- Der absolute Fehler misst die direkte Differenz zwischen exaktem Wert und Näherung.
- Der relative Fehler setzt diese Abweichung ins Verhältnis zur Größe von x .
- Dadurch erkennt man die prozentuale bzw. proportionale Abweichung.

Beispiel

Sei

$$x = 3,145, \quad \tilde{x} = 3,14.$$

Dann gilt:

$$|x - \tilde{x}| = |3,145 - 3,14| = 0,005.$$

Der absolute Fehler beträgt also:

$$0,005.$$

Weiter:

$$\left| \frac{x - \tilde{x}}{x} \right| = \frac{0,005}{3,145} \approx 0,00159.$$

Der relative Fehler beträgt also ungefähr:

$$0,159\%.$$

Bezug zur Maschinendarithmetik

Die Maschinengenauigkeit beschreibt den maximalen relativen Fehler, der beim Runden von Zahlen im darstellbaren Bereich auftreten kann.

Definition der Maschinengenauigkeit

Nun präzisieren wir den Begriff der Maschinengenauigkeit mathematisch.

Definition 5. Sei F ein Maschinenzahlbereich.

Die Maschinengenauigkeit von F wird mit

$$\varepsilon(F)$$

bezeichnet und definiert durch

$$\varepsilon(F) := \sup \left\{ \left| \frac{x - \text{rd}(x)}{x} \right| \mid x \in \mathbb{R}, |x| \in \text{range}(F) \right\}.$$

Dabei bezeichnet rd eine Rundung auf den Maschinenzahlbereich F .

Bedeutung der Formel

Die Definition beschreibt:

- Man betrachtet alle reellen Zahlen x , deren Betrag im darstellbaren Größenbereich liegt.
- Jede dieser Zahlen wird auf eine Maschinenzahl $\text{rd}(x)$ gerundet.
- Dann berechnet man den relativen Fehler

$$\left| \frac{x - \text{rd}(x)}{x} \right|.$$

- Das Supremum ist der größtmögliche relative Fehler, der auftreten kann.

Anschaulich

Die Maschinengenauigkeit ist also der schlechteste relative Rundungsfehler, den das Zahlensystem im normalen Darstellungsbereich verursachen kann.

Warum ist das wichtig?

- Sie misst die numerische Präzision eines Zahlensystems.
- Je kleiner $\varepsilon(F)$, desto genauer kann gerechnet werden.
- Viele Fehlerabschätzungen in der Numerik hängen direkt von $\varepsilon(F)$ ab.

Satz zur Maschinengenauigkeit

Satz 6. Für jeden Maschinenzahlbereich

$$F = F(b, m, E_{\min}, E_{\max}) \quad \text{mit} \quad E_{\min} < E_{\max}$$

gilt:

$$\varepsilon(F) = \frac{1}{1 + 2b^{m-1}}.$$

Bedeutung

Die Maschinengenauigkeit hängt nur ab von:

- der Basis b ,
- der Mantissenlänge m .

Sie hängt nicht direkt vom Exponentenbereich ab.

Warum ist $E_{\min} < E_{\max}$ sinnvoll?

Dann gibt es mindestens mehrere Größenordnungen von Zahlen. Der Maschinenzahlbereich ist also nicht degeneriert, sondern enthält einen echten Wertebereich.

Interpretation der Formel

- Größeres $m \Rightarrow$ kleinere Maschinengenauigkeit $\Rightarrow$ genaueres Rechnen.
- Größere Basis verändert den Abstand benachbarter Zahlen.

Beweisidee

Der größte relative Rundungsfehler entsteht:

- genau zwischen zwei benachbarten Maschinenzahlen,
- und zwar bei der kleinsten Größenordnung $b^{E_{\min}}$, weil dort derselbe absolute Fehler relativ am stärksten wirkt.

Der Abstand zweier benachbarter Zahlen dort beträgt

$$b^{E_{\min}} b^{-m+1}.$$

Der maximale absolute Fehler ist die Hälfte davon:

$$\frac{1}{2} b^{E_{\min}} b^{-m+1}.$$

Man betrachtet daher

$$x = b^{E_{\min}} \left(1 + \frac{1}{2} b^{-m+1} \right).$$

Dann ergibt sich:

$$\left| \frac{x - \text{rd}(x)}{x} \right| = \frac{\frac{1}{2} b^{E_{\min}} b^{-m+1}}{b^{E_{\min}} \left(1 + \frac{1}{2} b^{-m+1} \right)} = \frac{1}{1 + 2b^{m-1}}.$$

Damit ist das Supremum genau dieser Wert.

Warum darf man $\text{rd}(x) = b^{E_{\min}}$ wählen?

Für

$$x = b^{E_{\min}} \left(1 + \frac{1}{2} b^{-m+1} \right)$$

liegt x genau zwischen den beiden benachbarten Maschinenzahlen

$$a_1 = b^{E_{\min}} \quad \text{und} \quad a_2 = b^{E_{\min}} \left(1 + b^{-m+1} \right).$$

Denn der Abstand zwischen a_1 und a_2 beträgt

$$b^{E_{\min}} b^{-m+1},$$

und x liegt genau in der Mitte.

Also gilt:

$$|x - a_1| = |x - a_2|.$$

Damit kann je nach Rundungsregel entweder

$$\text{rd}(x) = a_1 = b^{E_{\min}}$$

oder

$$\text{rd}(x) = a_2$$

gelten.

Für den Fehlerbetrag ist das aber egal, da beide Abstände gleich groß sind. Deshalb darf man ohne Einschränkung bequem

$$\text{rd}(x) = b^{E_{\min}}$$

wählen.

Dann folgt:

$$\begin{aligned} \left| \frac{x - \text{rd}(x)}{x} \right| &= \left| \frac{b^{E_{\min}} (1 + \frac{1}{2} b^{-m+1}) - b^{E_{\min}}}{b^{E_{\min}} (1 + \frac{1}{2} b^{-m+1})} \right| \\ &= \frac{b^{E_{\min}} \cdot \frac{1}{2} b^{-m+1}}{b^{E_{\min}} (1 + \frac{1}{2} b^{-m+1})} = \frac{\frac{1}{2} b^{-m+1}}{1 + \frac{1}{2} b^{-m+1}} = \frac{1}{1 + 2b^{m-1}}. \end{aligned}$$

Daraus folgt:

$$\varepsilon(F) \geq \frac{1}{1 + 2b^{m-1}}.$$

$\Rightarrow \varepsilon(F)$ ist der größte mögliche relative Fehler.

Da wir bereits ein Beispiel gefunden haben, bei dem der relative Fehler

$$\frac{1}{1 + 2b^{m-1}}$$

auftritt, muss gelten:

$$\varepsilon(F) \geq \frac{1}{1 + 2b^{m-1}}.$$

Obere Abschätzung

Für die andere Richtung genügt es, positive Zahlen x zu betrachten, da wegen des Betrags negative Zahlen denselben relativen Fehler liefern.

Sei also

$$x \in \text{range}(F), \quad x \notin F.$$

Sei ferner

$$x = b^E \sum_{i=0}^{\infty} z_i b^{-i}$$

die normalisierte b -adische Darstellung von x .

Dann definieren wir die beiden benachbarten Maschinenzahlen

$$x' := b^E \sum_{i=0}^{m-1} z_i b^{-i},$$
$$x'' := b^E \left(\sum_{i=0}^{m-1} z_i b^{-i} + b^{-m+1} \right).$$

Dann gilt:

$$x', x'' \in F \quad \text{und} \quad x' < x < x''.$$

Also liegt die Rundung $\text{rd}(x)$ zwischen diesen beiden Nachbarzahlen.

Abschätzung des Rundungsfehlers

Da $\text{rd}(x)$ auf eine der beiden benachbarten Maschinenzahlen rundet, ist der Fehler höchstens halb so groß wie deren Abstand:

$$|x - \text{rd}(x)| \leq \frac{1}{2}(x'' - x').$$

Nun gilt:

$$x'' - x' = b^E \cdot b^{-m+1}.$$

Also folgt:

$$|x - \text{rd}(x)| \leq \frac{1}{2} b^E b^{-m+1}.$$

Beispiel: Für $b = 10$, $m = 3$, $x = 1,2424 \dots$ sind die Nachbarn

$$x' = 1,24, \quad x'' = 1,25.$$

Daher:

$$1,24 < x < 1,25$$

und der Rundungsfehler ist höchstens halb so groß wie der Abstand 0,01, also höchstens 0,005.

Da $z_0 > 0$ gilt, ist in der normalisierten Darstellung von x

$$x = b^E \sum_{i=0}^{\infty} z_i b^{-i}$$

die Mantisse mindestens 1. Also folgt:

$$x \geq b^E.$$

Da außerdem $b^E \in F$ eine Maschinenzahl ist und $\text{rd}(x)$ per Definition die Maschinenzahl mit minimalem Abstand zu x bezeichnet, gilt:

$$|x - \text{rd}(x)| \leq |x - b^E|.$$

Wegen $x \geq b^E$ ist

$$x = b^E + |x - b^E| \geq b^E + |x - \text{rd}(x)|.$$

Damit erhält man:

$$\left| \frac{x - \text{rd}(x)}{x} \right| \leq \frac{|x - \text{rd}(x)|}{b^E + |x - \text{rd}(x)|}.$$

Nun benutzen wir die bereits gezeigte Abschätzung

$$|x - \text{rd}(x)| \leq \frac{1}{2} b^E b^{-m+1}.$$

Einsetzen liefert:

$$\left| \frac{x - \text{rd}(x)}{x} \right| \leq \frac{1}{1 + \frac{b^E}{|x - \text{rd}(x)|}} \leq \frac{1}{1 + \frac{1}{\frac{1}{2}} b^E b^{-m+1}}.$$

Kürzen von b^E ergibt:

$$\left| \frac{x - \text{rd}(x)}{x} \right| \leq \frac{1}{1 + \frac{1}{\frac{1}{2}} b^{-m+1}} = \frac{1}{1 + 2b^{m-1}}.$$

Also gilt für alle $x \in \text{range}(F)$:

$$\left| \frac{x - \text{rd}(x)}{x} \right| \leq \frac{1}{1 + 2b^{m-1}}.$$

Daraus folgt:

$$\varepsilon(F) \leq \frac{1}{1 + 2b^{m-1}}.$$

Zusammen mit der bereits gezeigten unteren Abschätzung

$$\varepsilon(F) \geq \frac{1}{1 + 2b^{m-1}}$$

erhält man schließlich:

$$\varepsilon(F) = \frac{1}{1 + 2b^{m-1}}.$$

Damit ist Satz 4.8 bewiesen.

Beispiel: Maschinengenauigkeit von `double`

Für den IEEE-754-Typ `double` gilt:

$$b = 2, \quad m = 53.$$

Mit Satz 4.8 folgt:

$$\varepsilon(F_{\text{double}}) = \frac{1}{1 + 2 \cdot 2^{52}} = \frac{1}{1 + 2^{53}} \approx 1,11 \cdot 10^{-16}.$$

Das ist die Maschinengenauigkeit des Typs `double`.

Bedeutung

Bei korrekt gerundeten Operationen liegt der relative Rundungsfehler typischerweise höchstens in dieser Größenordnung.

Das entspricht ungefähr einer Genauigkeit von 15 bis 16 Dezimalstellen.

Unterschied zu `std::numeric_limits<double>::epsilon()`

In C++ bezeichnet

`std::numeric_limits<double>::epsilon()`

den Abstand von 1 zur nächsten darstellbaren Zahl größer als 1.

Für `double` ist das:

$$2^{-52} \approx 2,22 \cdot 10^{-16}.$$

Also gilt:

$$2^{-52} = 2 \cdot \varepsilon(F_{\text{double}}).$$

Die C++-Epsilon-Konstante ist also doppelt so groß wie die hier definierte Maschinengenauigkeit.

Definition signifikanter Stellen

Definition 7. Sei F ein Maschinenzahlbereich und $s \in \mathbb{N}$.

Eine Maschinenzahl $f \in F$ besitzt (mindestens) s signifikante Stellen in der b -adischen Gleitkommadarstellung, falls $f \neq 0$ gilt und für jede Rundung rd sowie jede Zahl $x \in \mathbb{R}$ mit

$$\text{rd}(x) = f$$

die Abschätzung

$$|x - f| \leq \frac{1}{2} b^{\lfloor \log_b |f| \rfloor + 1 - s}$$

erfüllt ist.

Bedeutung

Die Zahl f approximiert jede auf f gerundete Zahl x so genau, dass mindestens s führende Ziffern korrekt sind.

$$\lfloor \log_b |f| \rfloor$$

bestimmt dabei die Größenordnung von f , und der Exponent

$$\lfloor \log_b |f| \rfloor + 1 - s$$

markiert die Stelle der s -ten signifikanten Ziffer.

Beispiel zu signifikanten Stellen

Im Dezimalsystem ($b = 10$) sei eine gespeicherte Zahl

$$f = 1234$$

gegeben und es sollen $s = 3$ signifikante Stellen garantiert sein. Dann gilt:

$$\lfloor \log_{10}(1234) \rfloor = 3.$$

Also ist der Exponent in der Fehlerabschätzung:

$$3 + 1 - 3 = 1.$$

Damit fordert die Definition:

$$|x - f| \leq \frac{1}{2} \cdot 10^1 = 5.$$

Also dürfen alle Zahlen x im Intervall

$$[1229, 1239]$$

auf $f = 1234$ gerundet werden, ohne dass die ersten drei signifikanten Stellen verloren gehen.

Denn in diesem Bereich bleiben die führenden drei Ziffern gleich:

1, 2, 3.

Anschaulich

Die Definition gibt also eine klare Fehlergrenze an, sodass mindestens s signifikante Stellen zuverlässig korrekt bleiben.

Beispiel 4.11: Signifikante Stellen bei `double`

Jede Maschinenzahl

$$f \in F_{\text{double}} \setminus \{-\max F_{\text{double}}, 0, \max F_{\text{double}}\}$$

besitzt mindestens

$$\lfloor 52 \log_{10}(2) \rfloor = 15$$

signifikante Stellen in der Dezimaldarstellung.

Begründung

Der Datentyp `double` arbeitet binär mit 52 gespeicherten Mantissenbits (plus hidden bit). Die Binärpräzision lässt sich in Dezimalstellen umrechnen über:

$$\text{Dezimalstellen} \approx 52 \log_{10}(2).$$

Da

$$52 \log_{10}(2) \approx 15,65,$$

sind mindestens 15 sichere signifikante Dezimalstellen garantiert.

Bedeutung

Das heißt:

- Ein korrekt gespeicherter `double`-Wert ist in der Regel auf etwa 15 signifikante Dezimalstellen genau.
- Deshalb sieht man oft die Aussage: `double` hat ca. 15–16 Stellen Genauigkeit.
- Die 16. Stelle kann stimmen, muss aber nicht immer garantiert sein.

Beispiel

Für

$$f = 3,14159265358979$$

sind die ersten 15 signifikanten Stellen zuverlässig korrekt gespeichert.

Ausgabe in C++ mit 15 Stellen

Mit

```
#include <iomanip>
std::cout << std::setprecision(15);
```

kann man festlegen, dass bei der Ausgabe von Werten vom Typ `double` bis zu 15 signifikante Stellen angezeigt werden.

Bedeutung

Damit nutzt man die ungefähr garantierte Genauigkeit eines `double`-Werts sinnvoll aus.

Beispiel

```
double x = 3.141592653589793;
std::cout << std::setprecision(15) << x;
```

Ausgabe:

3.14159265358979

4 4.4 Maschinenzahlarithmetik

Sei $\circ$ eine der vier Operationen

$$\{+, -, \cdot, /\}.$$

Für $x, y \in F$ gilt im Allgemeinen nicht:

$$x \circ y \in F.$$

Das exakte mathematische Ergebnis ist also oft keine darstellbare Maschinenzahl.

Warum?

Gründe dafür sind zum Beispiel:

- Das Ergebnis besitzt unendlich viele Stellen (z. B. $1/3 = 0,333\dots$).
- Die Zahl liegt außerhalb des darstellbaren Bereichs (Overflow / Underflow).
- Es wären mehr Mantissenstellen nötig als gespeichert werden können.

Beispiele

$$0,1 + 0,2$$

ist binär nicht exakt darstellbar.

$$\frac{1}{3} = 0,333\dots$$

hat unendlich viele Stellen.

$$10^{300} \cdot 10^{300} = 10^{600}$$

kann außerhalb des Wertebereichs liegen.

Ersatzoperation auf dem Rechner

Darum verwendet ein Computer nicht die exakte Operation, sondern rechnet mit Rundung zurück in den Maschinenzahlbereich.

Wir schreiben dafür:

$$x \odot y \in F.$$

Annahme 4.12

Es gelte für eine Rundung rd zu F und alle $x, y \in F$:

$$x \odot y = rd(x \circ y).$$

Bedeutung

Der Rechner berechnet also:

1. exaktes Ergebnis $x \circ y$,
2. anschließend Rundung auf eine darstellbare Zahl.

Das gerundete Resultat ist dann wieder eine Maschinenzahl.

Hinweis zum IEEE-754-Standard

Die Eigenschaft

$$x \odot y = rd(x \circ y)$$

wird beispielsweise für die Grundoperationen des IEEE-754-Standards gefordert.

Dazu gehören insbesondere:

- Addition
- Subtraktion
- Multiplikation
- Division
- Quadratwurzel

Das bedeutet: Auch $\sqrt{x}$ soll korrekt gerundet als Maschinenzahl zurückgegeben werden.

Warum nicht für alle Funktionen?

Für Funktionen wie

$$\sin(x), \quad \cos(x), \quad \exp(x), \quad \log(x)$$

ist dies deutlich aufwendiger.

Sie werden meist nicht durch eine einzelne Hardware-Operation berechnet, sondern durch numerische Algorithmen, zum Beispiel:

- Polynomapproximationen
- Reihenentwicklungen
- Tabellenverfahren
- Bereichsreduktion

Darum ist korrektes Runden dort wesentlich komplizierter.

Praktischer Hinweis

Annahme 4.12 bedeutet, dass als Ergebnis einer Rechenoperation die gerundete nächstgelegene Maschinenzahl zum exakten Wert $x \circ y$ geliefert wird.

$$x \odot y = rd(x \circ y)$$

Wichtig

Das heißt jedoch nicht, dass der Rechner zuerst das exakte Ergebnis mit unendlich vielen Stellen vollständig berechnen muss.

In der Praxis genügt es, so viele Stellen zu berechnen, bis sicher entschieden werden kann, auf welche Maschinenzahl gerundet werden muss.

Dann kann die Berechnung abgebrochen werden.

Beispiel

Angenommen, nur zwei Dezimalstellen sind speicherbar und es ergibt sich:

$$x \circ y = 4,613728 \dots$$

Benachbarte Maschinenzahlen sind dann:

$$4,6 \quad \text{und} \quad 4,7.$$

Sobald klar ist, dass das Ergebnis näher an 4,6 liegt, muss nicht weitergerechnet werden.

Der Rechner kann direkt ausgeben:

$$x \odot y = 4,6.$$

Beispiel 4.13

Sei

$$F = F(10, 2, -5, 5).$$

Wir wählen

$$x = 4,5 \cdot 10^1 = 45, \quad y = 1,1 \cdot 10^0 = 1,1.$$

Es wird jeweils auf zwei Mantissenstellen gerundet.

Addition

$$x \oplus y = rd(x + y) = rd(46,1) = 4,6 \cdot 10^1.$$

Subtraktion

$$x \ominus y = rd(x - y) = rd(43,9) = 4,4 \cdot 10^1.$$

Multiplikation

$$x \otimes y = rd(x \cdot y) = rd(49,5).$$

Also je nach Rundungsregel:

$$x \otimes y \in \{4,9 \cdot 10^1, 5,0 \cdot 10^1\}.$$

(Bei IEEE-Rundung zur geraden Endziffer würde 50 entstehen.)

Division

$$x \oslash y = rd\left(\frac{x}{y}\right) = rd(40,9090 \dots) = 4,1 \cdot 10^1.$$

Bedeutung

Obwohl $x, y \in F$ gilt, liegen die exakten Ergebnisse meist nicht in F . Erst durch Rundung entstehen wieder Maschinenzahlen.

Relativer Fehler einer Rechenoperation

Gemäß Annahme 4.12 gilt:

$$x \odot y = rd(x \circ y).$$

Damit erhält man für den relativen Fehler:

$$\left| \frac{x \circ y - x \odot y}{x \circ y} \right| = \left| \frac{x \circ y - rd(x \circ y)}{x \circ y} \right|.$$

Falls

$$|x \circ y| \in \text{range}(F),$$

also das exakte Ergebnis im darstellbaren Bereich liegt, folgt aus der Definition der Maschinengenauigkeit:

$$\left| \frac{x \circ y - x \odot y}{x \circ y} \right| \leq \varepsilon(F).$$

Bedeutung

Der relative Rundungsfehler einer einzelnen Rechenoperation ist also höchstens so groß wie die Maschinengenauigkeit.

Je kleiner $\varepsilon(F)$, desto genauer rechnet das Zahlensystem.

Warum die Bedingung wichtig ist

Liegt $x \circ y$ nicht im Bereich $\text{range}(F)$, kann Overflow oder Underflow auftreten. Dann gilt diese Abschätzung nicht mehr direkt.

Rechengesetze in der Maschinenzahlarithmetik

Das Kommutativgesetz für Addition und Multiplikation gilt auch in der Maschinenzahlarithmetik.

Das Assoziativgesetz und das Distributivgesetz gelten im Allgemeinen jedoch nicht.

Der Grund ist, dass nach jedem Rechenschritt wieder auf eine Maschinenzahl gerundet wird. Dadurch verändern sich die Zwischenwerte, und mathematisch äquivalente Umformungen können zu unterschiedlichen Ergebnissen führen.

Beispiel 4.14

Im Maschinenzahlbereich

$$F = F(10, 2, -5, 5)$$

seien

$$x = 4,1 \cdot 10^0, \quad y = 8,2 \cdot 10^{-1}, \quad z = 1,4 \cdot 10^{-1}.$$

Dann gilt:

$$(x \oplus y) \oplus z = 4,9 \oplus 0,14 = 5,0$$

$$x \oplus (y \oplus z) = 4,1 \oplus 0,96 = 5,1$$

also ist die Addition nicht assoziativ.

Außerdem:

$$x \otimes (y \oplus z) = 4,1 \otimes 0,96 = 3,9$$

und

$$(x \otimes y) \oplus (x \otimes z) = 3,4 \oplus 0,57 = 4,0.$$

Damit ist auch das Distributivgesetz im Allgemeinen verletzt.

Erklärung

Warum passiert das?

Nach jedem einzelnen Rechenschritt wird gerundet:

$$\text{Rechnen} \longrightarrow \text{Runden} \longrightarrow \text{Weiterrechnen}.$$

Dadurch werden die Zwischenwerte verändert, und spätere Rechnungen laufen mit anderen Zahlen weiter als in der exakten Mathematik.

Folgerung für die Praxis

Mathematisch äquivalente Ausdrücke können in der Maschinenzahlarithmetik zu deutlich unterschiedlichen Ergebnissen führen, selbst wenn alle Eingabewerte darstellbare Maschinenzahlen sind.

Warum?

Ursachen sind insbesondere:

- Rundungsfehler,
- Reihenfolge der Rechenschritte,
- Auslöschungseffekte,
- Overflow / Underflow,
- Verlust signifikanter Stellen.

Bedeutung

Zwei Formeln können mathematisch exakt dasselbe bedeuten, aber auf dem Computer verschiedene Resultate liefern.

Mögliche Folgen

Dies kann in der Praxis ernste Auswirkungen haben, zum Beispiel:

- fehlerhafte Simulationen,
- ungenaue wissenschaftliche Ergebnisse,
- falsche Finanzberechnungen,
- Probleme in Technik oder Navigation,
- Fehler in medizinischer Software.

Darum spielt numerisch stabiles Rechnen eine zentrale Rolle.

Wichtigste Erkenntnis des Kapitels

In der numerischen Mathematik zählt nicht nur, *ob* eine Formel mathematisch richtig ist, sondern auch, *wie* sie auf dem Computer berechnet wird.

Warum?

Durch endliche Genauigkeit, Rundungen und Reihenfolge der Rechenschritte kann dieselbe mathematische Formel zu unterschiedlichen numerischen Ergebnissen führen.

Merksatz

Mathematisch korrekt $\neq$ numerisch zuverlässig.

Darum sucht die numerische Mathematik nicht nur richtige, sondern auch stabile und genaue Rechenverfahren.